
UNMARK

Tone-Factored Input Adaptation for
Diacritic-Robust Vietnamese Language Understanding

Research Proposal

Version 1.0 — 18 August 2026

Author	Quoc-Linh Vo (Võ Quốc Linh)
Affiliation	Faculty of Information Technology University of Science, VNU-HCM (APCS)
Contact	vqlinh2433@apcs.fitus.edu.vn
Target venue	SOICT 2026, Track 4 — AI Foundations, Foundation Models, and Generative AI
Key dates	Abstract 09 Sep 2026 · Full paper 16 Sep 2026 Notification 12 Oct · Camera-ready 23 Oct Conference 4–5 Dec 2026, Ho Chi Minh City
Format	Springer CCIS/LNCS, max 12 pages, single-blind

This document is the working specification for the project. It records the idea, its exact position relative to prior work, the method, the experimental design, the implementation plan, and the decision gates at which the project should be abandoned or redirected.

Executive summary

Vietnamese users routinely type without diacritics, yet every pretrained Vietnamese language model is trained and evaluated on fully diacritized text. The standard responses are to *restore* the missing marks at the string level, or to *measure* how much performance degrades when they are absent.

UNMARK asks a different question: can a frozen pretrained Vietnamese encoder be adapted to operate *directly* under structured orthographic information loss — without restoring the text, without changing the tokenizer, and without updating the encoder?

The proposed answer is a small input-side module that re-expresses Vietnamese orthography as structured channels: an unmarked base channel, a tone channel, and a letter-diacritic channel. The design is inspired by multi-source partial-diacritic modelling for Arabic, but Vietnamese breaks a core assumption of that formulation. In Arabic, the absence of a diacritic reliably signals *unavailable information*. In Vietnamese it does not: the *ngang* tone is itself written with no mark at all, so an unmarked syllable is simultaneously a valid observation and a possible information loss. UNMARK therefore models an UNMARKED state that is deliberately ambiguous, and lets the encoder resolve it from context.

The evaluation is organised around a question that has not been answered for Vietnamese: *at which level should structured orthographic loss be handled* — the string, the input representation, or the output representation? Three interventions are compared under one backbone, one task suite, and one controlled corruption protocol.

Contribution type. This is a *combination* contribution plus a new problem framing, not a new mechanism. Section 3 states explicitly what is and is not claimed. The project is calibrated for a mid-tier venue and should not be presented as more than that.

Contents

Executive summary	1
1 Background and motivation	2
1.1 Vietnamese tone orthography	2
1.2 The unmarked-tone ambiguity	2
1.3 Why this matters in deployment	2
1.4 Why existing responses are insufficient	3
2 Research question and hypotheses	3
3 Related work and positioning	3
3.1 The five nearest lines of work	3
3.2 What is <i>not</i> claimed	4
3.3 What <i>is</i> claimed	4
3.4 Positioning sentence for the paper	5
4 Method	5
4.1 Notation	5
4.2 Deterministic decomposition	5
4.3 Channel state sets	5
4.4 Alignment between streams	6
4.5 Fusion and gating	6
4.6 Training objectives	6
4.7 Parameter budget	7

5	Experimental design	7
5.1	Backbones	7
5.2	Tasks	7
5.3	Corruption conditions	7
5.4	Systems compared	8
5.5	Metrics	8
5.6	Controls	8
6	Feasibility gates	9
7	Implementation plan	9
7.1	Repository layout	9
7.2	Core interfaces	10
7.3	Two-stage training	10
7.4	Compute	10
8	Timeline	10
9	Risks and mitigations	11
10	Reproducibility and artifact	11
11	Fallback: the analysis-only paper	12
12	Open decisions	12

1 Background and motivation

1.1 Vietnamese tone orthography

Vietnamese is written in a Latin-based script (*Quốc ngữ*) in which each syllable follows a fixed phonotactic template C_1wVC_2T : onset, medial, nucleus, coda, and tone. The inventory is closed and small — on the order of 21 onsets, 155 finals, and 6 tones — yielding roughly 7,000 syllables in practical use out of about 19,000 that are phonotactically possible. This regularity means a Vietnamese syllable can be decomposed into its components by rule, deterministically, with no model and no lookup table.

Two distinct classes of diacritic must be separated, because they carry different information and can be lost independently:

- **Tone marks** realise five of the six lexical tones: *sắc, huyền, hỏi, ngã, nặng*.
- **Letter diacritics** form distinct vowel and consonant letters: *ă, â, ê, ô, ó, ù, đ*. These belong to the letter identity, not to the tone.

Tone is lexically contrastive: changing the tone changes the word, not its register or nuance. A single base syllable can correspond to several unrelated words.

1.2 The unmarked-tone ambiguity

The sixth tone, *ngang*, is written by placing *no mark at all*. This is the linguistic fact on which the whole project turns.

Consequently, when a system observes a syllable carrying no tone mark, the orthography alone does not distinguish two very different situations:

Situation	Meaning of “no tone mark”
Genuine <i>ngang</i>	The tone <i>is ngang</i> . Information is present.
Stripped input	The tone was one of five others; the mark was lost.

Both surface as an identical string. The ambiguity is *not observable* at inference time, and no amount of engineering removes it — it is a property of the writing system.

This is where Vietnamese departs from the languages in which partial-diacritic modelling was developed. In Arabic, ordinary text is written undiacritized, so absence of a mark means “not yet specified” — a single, unambiguous state that a dedicated blank symbol represents faithfully. In Vietnamese, the same symbol would conflate a meaningful phonological category with an information loss.

1.3 Why this matters in deployment

Undiacritized Vietnamese is not an edge case. It is how a large share of real-world Vietnamese input is produced: search boxes, chat, comments, quick notes, and any setting where typing speed matters. It also arises mechanically, as OCR systematically drops tone marks — reported error taxonomies for Vietnamese scene text place tone-drop at roughly a fifth of all recognition errors, rising further when recogniser confidence is low.

Meanwhile, the pretrained models used to process this input were trained on Wikipedia and news text, which is fully diacritized. Prior work has already established that Vietnamese PLM performance degrades substantially as diacritics are removed, and that subword tokenizers fragment undiacritized Vietnamese badly. The distribution mismatch is documented; what is missing is an inexpensive way to *operate* under it.

1.4 Why existing responses are insufficient

String-level restoration. Predict the missing marks, then encode the repaired string. Vietnamese diacritic restoration is a mature area with high reported token-level accuracy. Two concerns remain untested. First, restoration must *commit* to a single output; where the preimage is genuinely ambiguous, a wrong commitment is unrecoverable downstream. Second, reported accuracies are measured on well-formed running text; short, context-poor inputs are a different distribution. Whether high token accuracy translates into recovered downstream performance is an open empirical question — and answering it is part of this project.

Output-level representation alignment. Encode the corrupted string as usual, then learn a light transformation pulling the resulting vector towards the clean one, with the encoder frozen. This family is well established for typographical noise. It repairs the consequence after tokenization has already fragmented the input.

Retraining. Pretrain a model on data that includes undiacritized text. Effective, and demonstrably improves robustness — but it costs a pretraining run, and it does not help the many teams already running an existing encoder in production.

UNMARK occupies the remaining position: intervene at the *input representation*, before the encoder, leaving the encoder untouched.

2 Research question and hypotheses

Primary research question

RQ. Can a pretrained Vietnamese language model be made robust to structured orthographic information loss without restoring the text, changing the tokenizer, or updating the pretrained encoder?

Secondary research question

RQ'. At which level should structured orthographic loss be handled: the string, the input representation, or the output representation?

Hypotheses

H1 (architectural feasibility). A frozen pretrained encoder accepts a synthesised input embedding built from orthographic channels without substantial loss on fully diacritized input.

H2 (residual gap). Even with a strong off-the-shelf diacritic restorer, a measurable downstream gap remains between restored input and fully diacritized input.

H3 (input-level advantage). Channel-factored input adaptation recovers a larger fraction of that gap than output-level representation alignment, at comparable or lower parameter and latency cost.

H4 (ambiguity modelling). Explicitly modelling the UNMARKED state as ambiguous outperforms treating it as a single “missing” symbol, and outperforms treating it as a confident *ngang*.

H2 is the load-bearing hypothesis: if it fails, the project has no method-paper story. H4 is the hypothesis carrying whatever originality the work has. Both are tested early (Section 6).

3 Related work and positioning

3.1 The five nearest lines of work

Multi-source partial-diacritic modelling (Arabic). Bahar et al. (2023) propose a two-source model in which an undiacritized character stream and a diacritic stream are embedded separately

and summed, with a blank symbol for positions carrying no diacritic information, trained with random masking of the diacritic channel at varying rates. This is the closest ancestor of UNMARK. Its *output* is diacritics; UNMARK’s output is a semantic representation. Its blank symbol is unambiguous; UNMARK’s UNMARKED state is not.

Vietnamese PLMs and diacritic degradation. ViSoBERT (2023) pretrains a Vietnamese social-media encoder and, as part of its analysis, measures degradation of Vietnamese PLMs under 25/50/75/100% diacritic removal across several classification tasks, and documents tokenizer fragmentation on undiacritized text. This is the empirical baseline UNMARK builds on — it establishes the problem, and it means degradation curves are not by themselves a contribution.

Deterministic Vietnamese phonological decomposition. PhonoSTFG (2026) performs NFD normalisation, tone extraction, tone stripping, and rule-based syllable parsing for Vietnamese, and uses the result as pairwise binary features forming an attention bias inside a graph, on top of a frozen PhoBERT with a per-dimension gated fusion of two streams. The decomposition procedure is reused here. The use is different: pairwise comparison between tokens versus per-token state representation at the input layer.

Factorized orthographic input representations. Work on decomposing accented characters into base plus modifier for machine translation across many languages, and recent character-level rich embeddings intended as drop-in replacements for subtoken embeddings, both argue that factorisation mitigates sparsity caused by subword tokenization. UNMARK inherits this argument and must not restate it as new.

Noise-robust dense retrieval. Contrastive alignment of noisy-query representations to clean ones with a frozen index, ranking-distillation approaches, and multi-positive variants. These operate at the output of the encoder and form the principal competing baseline.

3.2 What is *not* claimed

Stating these plainly in the paper is deliberate. A reviewer who sees the boundaries drawn honestly is more likely to accept what lies inside them.

- N1. Not the first to decompose a syllable into base and diacritic components.
- N2. Not the first to treat diacritics as a separate input channel that may be absent.
- N3. Not the first to train with randomly masked diacritics at multiple rates.
- N4. Not the first to measure Vietnamese downstream degradation under diacritic removal.
- N5. Not claiming that subword fragmentation is eliminated. UNMARK makes the diacritized and undiacritized forms share a base sequence; it does not make that sequence well tokenized.
- N6. Not claiming cross-lingual transfer to other diacritic-bearing orthographies. The formulation is potentially portable; validation is left to future work.
- N7. Not claiming Vietnamese is the “most extreme” case of orthographic ambiguity. It is an *informative* case, for the specific reason given in Section 1.2.

3.3 What *is* claimed

- C1. **A reframed objective.** Prior work on diacritics either restores them or measures the damage from their absence. UNMARK adapts a pretrained model to operate under the loss, targeting representation stability rather than orthographic recovery.
- C2. **Ambiguity-aware channel state.** The tone channel encodes only what is observable. UNMARKED deliberately conflates genuine *ngang* with stripped tone, and the training signal — in which *ngang* syllables are invariant under corruption while others are not — teaches the model that this state is ambiguous and must be resolved from context. This asymmetry has no analogue in the Arabic formulation.
- C3. **Parameter-efficient retrofit.** A reusable module for existing frozen Vietnamese encoders: no pretraining, no tokenizer change, no index rebuild.

C4. A three-level intervention comparison. String-level restoration, input-level factorisation, and output-level alignment, evaluated under one backbone, one task suite, one corruption protocol.

3.4 Positioning sentence for the paper

Existing work either restores missing diacritics or measures the degradation their removal causes. We instead ask whether a pretrained model can be adapted to operate directly under structured orthographic missingness. Our design is inspired by multi-source partial-diacritic modelling such as 2SDiac, but Vietnamese introduces a fundamentally different ambiguity: the absence of a tone mark simultaneously encodes the phonologically meaningful ngang tone.

4 Method

4.1 Notation

Let x be a fully diacritized Vietnamese string and $\tilde{x}_p$ a corrupted version in which tone marks are removed from a fraction p of syllables. Let E_θ be a pretrained encoder with frozen parameters θ , Emb_θ its input embedding table, and $h(\cdot)$ the representation it produces (pooled or per-token as the task requires). Let A_ϕ denote the UNMARK module with trainable parameters ϕ , $|\phi| \ll |\theta|$.

4.2 Deterministic decomposition

A rule-based function maps a string to three parallel syllable-level streams:

$$\text{dec}(x) = (b(x), \tau(x), \lambda(x))$$

where b is the base form with all diacritics removed, τ is the tone state, and λ is the letter-diacritic state. The procedure is: Unicode NFD normalisation; separation of combining marks into tone marks (U+0300, U+0301, U+0303, U+0309, U+0323) and letter-forming marks (U+0302, U+0306, U+031B, plus the đ stroke); recomposition of the base.

Invertibility. dec must satisfy $\text{rec}(\text{dec}(x)) = x$ exactly, for every x , including punctuation, digits, Latin loanwords, and mixed-script tokens. This is verified by an exhaustive round-trip test (Section 6, G0). Silent Unicode loss here would contaminate every downstream number without raising an error.

4.3 Channel state sets

Channel	States	Notes
Base b	subword ids of the stripped form	via the frozen tokenizer
Tone τ	$\{\text{UNMARKED}, \text{sắc}, \text{huyền}, \text{hỏi}, \text{ngã}, \text{nặng}\}$	6 states
Letter λ	$\{\text{NONE}, \text{breve}, \text{circumflex}, \text{horn}, \text{stroke}, \dots\}$	small closed set
Non-Vietnamese	N/A	digits, Latin words, symbols

Design decision: no separate MISSING state. An earlier draft used six tones plus a MISSING symbol. This is not implementable: at inference the system observes only *no visible mark*, and cannot know whether a mark was deleted. Any state set requiring that knowledge works only in the laboratory.

The tone channel therefore encodes *only what is observable*. Genuine *ngang* and stripped tone both map to UNMARKED. This conflation is intentional and is the technical core of the proposal:

- Under corruption, a *ngang* syllable is *invariant*; a toned syllable transitions to UNMARKED. The training distribution therefore contains UNMARKED tokens of both origins, in a ratio that varies with the corruption rate.
- The module cannot resolve the ambiguity from the channel value alone. It must learn that UNMARKED is a low-information state whose interpretation depends on the surrounding context.
- This is precisely the behaviour a blank-symbol formulation does not need to learn, because in the source language the blank is unambiguous.

4.4 Alignment between streams

This is the most error-prone engineering detail and must be settled before any training run.

Three streams do not share an index. The original string, the base string, and the syllable-level channel labels tokenize differently: $|T(x)| \neq |T(b(x))|$ in general, and one syllable may span several subwords.

The specification is:

1. The **base stream** defines the token grid. All positions are indexed by $T(b(x))$.
2. Each subword is assigned the channel labels of the syllable it belongs to, by tracking character offsets through tokenization.
3. The **original stream** is *not* used as a third parallel stream, because it cannot be aligned to the base grid without an ad-hoc heuristic. If a residual path to the original embedding is wanted, it is added only in the fully diacritized condition, where $b(x)$ and x share the syllable segmentation.

4.5 Fusion and gating

For each position i on the base grid:

$$\begin{aligned} e_i &= \text{Emb}_\theta(b_i), & t_i &= W_\tau[\tau_i], & l_i &= W_\lambda[\lambda_i] \\ f_i &= \text{LN}(W_f[e_i; t_i; l_i] + c_f) \\ g_i &= \sigma(W_g[e_i; t_i; l_i] + c_g) \in (0, 1)^d & z_i &= g_i \odot f_i + (1 - g_i) \odot e_i \end{aligned}$$

The vector z_i replaces the ordinary input embedding. All transformer layers above remain frozen.

The gate provides a structural guarantee: $g_i \rightarrow 0$ recovers the unmodified model exactly. This matters because in deployment the system does not know whether the input is diacritized. Note that gating on a frozen encoder is standard practice and is *not* claimed as novel; it is included because the identity-recovery property is needed, and it must be cited accordingly.

4.6 Training objectives

Training is self-supervised and requires no annotation. Any Vietnamese corpus suffices; corrupted counterparts are generated by rule.

Structured channel dropout. For each example, sample a corruption rate $p \sim \mathcal{U}(0, 1)$ and set the tone channel to UNMARKED for a random p -fraction of syllables. Sampling p continuously, rather than using a fixed rate or only the endpoints, is essential: a model trained only at $p = 0$ and $p = 1$ will behave poorly at intermediate rates, which are the realistic ones. An optional second rate governs letter-diacritic dropout.

Alignment loss. Match the corrupted representation to the clean one produced by the same frozen encoder without the module:

$$\mathcal{L}_{\text{align}} = D(h'(\tilde{x}_p), h(x))$$

with D a cosine or smooth- L_1 distance over pooled and/or per-token representations.

Clean-preservation loss. Force near-identity behaviour on uncorrupted input:

$$\mathcal{L}_{\text{clean}} = D(h'(x), h(x))$$

Total. $\mathcal{L} = \lambda_a \mathcal{L}_{\text{align}} + \lambda_c \mathcal{L}_{\text{clean}}$, with λ_a, λ_c tuned on a development split. The measurable training objective, stated conservatively, is

$$D(h_{\text{UNMARK}}(x), h_{\text{UNMARK}}(\tilde{x})) < D(h_{\text{base}}(x), h_{\text{base}}(\tilde{x})).$$

4.7 Parameter budget

Component	Parameters
Tone embedding table ($6 \times d$)	$\approx 5\text{K}$
Letter-diacritic table ($\sim 8 \times d$)	$\approx 6\text{K}$
Fusion projection ($3d \rightarrow d$)	$\approx 1.8\text{M}$
Gate projection ($3d \rightarrow d$)	$\approx 1.8\text{M}$
Layer norms	$\approx 2\text{K}$
Total trainable	$\approx 3.6\text{M}$
Frozen encoder	135M+

A linear fusion variant reduces this further and is included in the ablation.

5 Experimental design

5.1 Backbones

At least two, to show the module is not tied to one model: PhoBERT-base (word/syllable-level BPE, trained on formal diacritized text) and ViSoBERT (social-media oriented, already partially robust). A multilingual encoder such as XLM-R is a third option if time allows. Including ViSoBERT is deliberate: it is the hardest case, since it is already the most robust baseline available, and a gain there is the most convincing result obtainable.

5.2 Tasks

Classification, not retrieval — substantially cheaper and sufficient to test every hypothesis. Use the task suite on which Vietnamese diacritic degradation has already been reported, so numbers are comparable to published values: emotion recognition, hate-speech detection, sentiment analysis, and spam-review detection.

5.3 Corruption conditions

Condition	Description
FULL	Fully diacritized (upper bound)
P25 / P50 / P75	Tone marks removed from 25/50/75% of syllables
P100	All tone marks removed
STRIP-ALL	Tone and letter diacritics removed (real typing behaviour)
VARIANT	Tone-placement variants (hoà/hòa) and NFC/NFD forms

STRIP-ALL is the condition that matches how people actually type and should be reported as the headline number. VARIANT is cheap to add and tests a failure mode nobody has measured.

5.4 Systems compared

System	Level	Description
UPPER	—	Clean input, unmodified model
FLOOR	—	Corrupted input, unmodified model
RESTORE	String	Off-the-shelf diacritic restorer, then encode
ALIGN	Output	Contrastive alignment adapter on the encoder output
UNMARK	Input	This proposal

All five share the backbone, the classification head architecture, the hyperparameters, the seeds, and the training budget. Any deviation makes the comparison uninterpretable.

5.5 Metrics

Task level. Macro-F1 and accuracy per task and condition.

Gap Recovery Rate. The headline number, normalising across tasks of different difficulty:

$$\text{GRR} = \frac{S_{\text{system}} - S_{\text{FLOOR}}}{S_{\text{UPPER}} - S_{\text{FLOOR}}}$$

Reported for RESTORE, ALIGN, and UNMARK. The comparison of these three GRR values is the result of the paper.

Representation level. Cosine similarity between clean and corrupted representations, before and after the module; the distribution shift; and a low-dimensional projection showing convergence of the two populations. This is what distinguishes an analysis from a leaderboard entry.

Cost. Added parameters, encoding latency, and whether the pipeline requires an additional model at inference. RESTORE carries an autoregressive decoder; UNMARK carries a matrix multiplication.

Diagnostic. Restorer token-level accuracy measured separately on long text and on short, context-poor input, and error rate broken down by word class (proper nouns and domain terms versus function words). If the predicted pattern holds — errors concentrated in the high-information words — this is a single figure that motivates the entire paper.

5.6 Controls

- **No clean-input regression.** Every system must be reported on FULL. A system that improves corrupted input while degrading clean input is useless in deployment, where the mode is unknown.
- **Seeds.** At least three per configuration; report mean and standard deviation. If the improvement is within seed variance, there is no result, and this must be stated.
- **Ablations.** Linear versus MLP fusion; with versus without gate; tone channel only versus tone plus letter channel; UNMARKED-as-ambiguous versus a forced *ngang* assignment (this is the direct test of H4); training data volume; fixed versus sampled corruption rate.
- **LLM reference point.** A zero-shot general-purpose LLM row, included as a reference rather than a competing baseline, to pre-empt the “why not just use an LLM” question and to test whether the problem disappears at scale.

6 Feasibility gates

These run *before* any writing. Their purpose is to fail fast.

G0 — Decomposition correctness (Day 1). Implement `decompose` and `recompose`; run round-trip on $\geq 100\text{K}$ sentences of mixed Vietnamese.

Pass: exact match on 100% of inputs. Not 99%.

Fail: fix before proceeding. Unicode loss here is silent and poisons everything downstream.

G1 — Frozen encoder acceptance (Day 2). Attach the fusion layer, train briefly on a small corpus, force the gate towards identity, evaluate on one classification task with FULL input.

Pass: within ≈ 1 point of the unmodified model.

Fail: the encoder rejects the synthesised embedding distribution. Options: unfreeze the lowest layers (loses the retrofit claim), or abandon the input-level design. Either way, know it on day 2.

G2 — Residual gap after restoration (Day 3). Run UPPER, FLOOR, and RESTORE on two tasks.

Pass: a clear gap remains between RESTORE and UPPER.

Fail: if restoration nearly closes the gap, H2 is false. The paper becomes “a cheaper route to equivalent performance” — publishable but much weaker — or the project falls back to the analysis-only version (Section 11).

Three days to decide whether to invest the remaining eighteen. This is the best trade available in the project.

7 Implementation plan

7.1 Repository layout

```
unmark/
  README.md
  requirements.txt
  configs/
    base.yaml # backbone, paths, seeds
    train_module.yaml # stage 1: self-supervised module training
    train_head.yaml # stage 2: task head training
    ablations/ # one file per ablation
  unmark/
    __init__.py
    orthography/
      decompose.py # dec / rec, channel state sets
      corrupt.py # structured channel dropout
      variants.py # NFC/NFD, tone-placement variants
      tables.py # syllable inventory, mark inventories
    align/
      offsets.py # syllable -> subword label propagation
    modules/
      channels.py # tone & letter embedding tables
      fusion.py # fusion projection + gate
      unmark.py # wraps a frozen encoder
    baselines/
      restore.py # string-level restoration wrapper
      align_adapter.py # output-level alignment adapter
    training/
      stage1_module.py
      stage2_head.py
      losses.py
    eval/
      metrics.py # macro-F1, GRR
      representation.py # cosine, shift, projection
      cost.py # latency, parameter counts
      report.py # builds result tables
    scripts/
      g0_roundtrip.py # gate 0
      g1_acceptance.py # gate 1
      g2_restore_gap.py # gate 2
```

```

run_grid.py
tests/
test_decompose.py # round-trip, edge cases
test_offsets.py # alignment invariants
results/
paper/

```

7.2 Core interfaces

```

# unmark/orthography/decompose.py
@dataclass
class Decomposition:
    base: str # all diacritics removed
    tone: list[int] # per syllable; 0 = UNMARKED
    letter: list[int] # per syllable; 0 = NONE
    spans: list[tuple[int,int]] # char span of each syllable in 'base'

def decompose(text: str) -> Decomposition: ...
def recompose(d: Decomposition) -> str: ... # must invert exactly

# unmark/orthography/corrupt.py
def strip_tones(d: Decomposition, p: float, rng) -> Decomposition: ...
def strip_letters(d: Decomposition, p: float, rng) -> Decomposition: ...

# unmark/align/offsets.py
def propagate(d: Decomposition, tokenizer) -> tuple[list[int], list[int], list[int]]:
    """Return (input_ids, tone_ids, letter_ids) on the BASE token grid."""

# unmark/modules/unmark.py
class UnmarkEncoder(nn.Module):
    """Frozen encoder + trainable input-side module."""
    def __init__(self, encoder, d_model, n_tone=6, n_letter=8,
                 fusion="mlp", use_gate=True): ...
    def forward(self, input_ids, tone_ids, letter_ids, attention_mask): ...

```

7.3 Two-stage training

1. **Stage 1 — module.** Self-supervised. Encoder frozen. Train ϕ with $\mathcal{L}_{\text{align}} + \mathcal{L}_{\text{clean}}$ under sampled corruption. No labels required. Corpus: any Vietnamese text.
2. **Stage 2 — head.** Module frozen, encoder frozen. Train a classification head per task. Run identically for all five systems.

Keeping the stages separate is what makes the comparison fair: only the input representation differs between systems at stage 2.

7.4 Compute

One GPU. Stage 1 is a few hours; the full stage-2 grid (5 systems $\times$ tasks $\times$ conditions $\times$ 3 seeds) is the larger cost and should be scripted, checkpointed, and resumable from the start. Colab or Kaggle is sufficient.

8 Timeline

Today is 18 August 2026. The full-paper deadline is 16 September 2026.

Dates	Phase	Deliverable
Aug 18–20	Gates	G0, G1, G2 passed or project redirected
Aug 21–26	Build	Module, corruption pipeline, both baselines, stage-1 training
Aug 27–31	Grid	Full experimental grid, 3 seeds, 2 backbones
Sep 1–4	Analysis	Ablations, representation analysis, diagnostic figures
Sep 5–8	Draft	Full draft; abstract finalised
Sep 9	Milestone	Abstract submitted
Sep 9–14	Revise	Related work hardened; claims audited against Section 3
Sep 15	Buffer	Reserved for what goes wrong
Sep 16	Milestone	Full paper submitted

The buffer day is not optional. Every deadline project needs one, and it should not be spent in advance.

9 Risks and mitigations

Risk	Sev.	Mitigation
Frozen encoder rejects synthesised embeddings	High	G1 on day 2; fallback to unfreezing lowest layers or to the analysis paper
Restoration closes the gap (H2 false)	High	G2 on day 3; pivot the story to cost, or fall back
Reviewer reads the work as “2SDiac for Vietnamese”	High	Confront it in the introduction; H4 ablation is the concrete rebuttal
Machinery overlaps PhonoSTFG (frozen PLM + gate)	Medium	Cite in the method section, not only in related work
Gains within seed variance	Medium	Three seeds from the start; report honestly if so
Alignment bug between streams	Medium	Unit tests and invariants before training (Section 4.4)
Unicode edge cases (NFC/NFD, mixed script)	Medium	G0 round-trip on 100K sentences
Concurrent Vietnamese work not indexed in English	Medium	Search Vietnamese sources and SOICT/RIVF/K-SE/NICS proceedings early
Scope creep into decoding-side work	Low	Explicitly out of scope; a neighbouring group has flagged that direction

10 Reproducibility and artifact

The module is the artifact, and it only becomes one if released properly. Plan from day one, not at submission:

- Public repository with code, configs, and seeds.
- Trained module weights for each backbone — a few megabytes.
- The decomposition and corruption utilities as a standalone, importable component, so others can reuse them without adopting the model.
- A minimal usage example: load a frozen encoder, wrap it, run inference.
- Exact dataset versions and splits; all preprocessing scripted.
- A results directory containing raw per-seed numbers, not only aggregates.

11 Fallback: the analysis-only paper

If G1 or G2 fails, the project pivots rather than dies. The fallback paper:

A controlled study of how existing robustness strategies behave under Vietnamese diacritic loss — a structured, ambiguous form of orthographic degradation that differs from the character noise these methods were designed for. String-level restoration, output-level alignment, and augmentation-based robustness are compared across corruption rates, backbones, and tasks, with error analysis by word class and a diagnosis of where each strategy breaks.

This is honest, feasible in the remaining time, reuses everything already built, and is publishable at the target venue. It should be prepared as a genuine option, not treated as failure.

12 Open decisions

To be settled during the gate phase and recorded here as they are made:

1. One channel for all diacritics, or separate tone and letter channels? (Current specification: separate. Ablation will test it.)
2. Pooled-representation alignment, per-token alignment, or both?
3. Which diacritic restorer serves as the RESTORE baseline, and how is its own accuracy reported?
4. Corpus for stage-1 training: size, domain mix, and whether it should match the downstream task domains.
5. Whether to include a retrieval task if time permits, or stay entirely within classification.

References

- [1] P. Bahar, M. Di Gangi, N. Rossenbach, M. Zeineldeen. Take the Hint: Improving Arabic Diacritization with Partially-Diacritized Text. *Interspeech*, 2023.
- [2] N. Q. Nguyen et al. ViSoBERT: A Pre-Trained Language Model for Vietnamese Social Media Text Processing. *EMNLP*, 2023.
- [3] D. Q. Nguyen, A. T. Nguyen. PhoBERT: Pre-trained Language Models for Vietnamese. *Findings of EMNLP*, 2020.
- [4] N. N.-Y. Nguyen, A.-D. Nguyen, N. H. Nguyen, K. V. Nguyen, N. L.-T. Nguyen. Linguistically Informed Multimodal Fusion for Vietnamese Scene-Text Image Captioning: Dataset, Graph Framework, and Phonological Attention. arXiv:2604.27712, 2026.
- [5] Noise-Robust Dense Retrieval via Contrastive Alignment Post Training, 2023.
- [6] Typo-Robust Representation Learning for Dense Retrieval. *ACL*, 2023.
- [7] S. Zhuang, G. Zuccon. Characterbert and Self-Teaching for Improving the Robustness of Dense Retrievers on Queries with Typos. *SIGIR*, 2022.
- [8] Z. Sun et al. ChineseBERT: Chinese Pretraining Enhanced by Glyph and Pinyin Information. *ACL*, 2021.
- [9] Interplay of Machine Translation, Diacritics, and Diacritization. *NAACL*, 2024.
- [10] L. S. T. Nguyen, T. T. Quan. Which Works Best for Vietnamese? A Practical Study of Information Retrieval Methods across Domains. *Findings of EACL*, 2026.

Citations above are working entries recorded during the literature survey. Full bibliographic details must be verified against the official proceedings before submission.